



# 教學報告：放洋全端 CRM 平台 — Vibe Coding × 真人工程師的 AI 協作開發實錄

這份報告是什麼。記錄放洋「全端 CRM 平台」從一個營運需求，一路做到通過業界資安檢測、準備移交資訊部的完整協作過程。重點不在炫耀成果，而在拆解「一個不寫程式的人，如何靠 AI 協作把一套含大量個資的系統做到資安標準」這件事是怎麼運轉的，作為日後其他專案的可複製範本。

讀者：不限工程背景。所有技術名詞都會用白話解釋。

一句話總結：一位營運負責人（Marcus）＋一個負責規劃與文件的 AI（Claude）＋一個負責實際動手的 AI（Claude Code），加上扮演「驗收方／訂標準」的資訊部，靠一套嚴謹的「AI 規劃 → AI 執行 → 真人驗收」循環，把系統做到四項資安評級全 A。

## 一、先搞懂幾個名詞（給非技術讀者）

- **Vibe coding（氛圍編程）**：指「人不直接寫程式碼，而是用自然語言描述想要什麼，由 AI 產生程式、人負責驗收行為與結果」的開發方式。重點是人掌握『**要做什麼**』與『**做得對不對**』，把『**怎麼寫**』交給 AI。本專案就是典型案例。
- **CRM**：客戶關係管理系統。放洋這套是管理留學／遊學學生「從名單建立 → 諮詢成交 → 文件 → 申請 → 入學」的核心系統，裡面有大量學生個資與各平台帳密。
- **Claude（本對話） vs Claude Code（CC）**：兩個都是 AI，分工不同。**Claude** 是對話視窗裡這個，負責思考、規劃、寫文件、做資安推理，但**不直接碰你的電腦**。**Claude Code** 是裝在電腦上、能操作真實終端機（安裝工具、跑指令、改檔案、commit）的 AI，是實際動手的那隻。
- **Notion**：團隊文件平台。本專案所有交付文件（8 份主文件 + 2 份附件）都放在這裡。

- **資安掃描工具**：自動檢查程式有沒有漏洞的軟體。本專案用了兩套——**SonarQube (SonarCloud)** 是「評級制」，會給 A～E 的等第；**Semgrep** 是「深度規則掃描」，會逐條列出可疑寫法。
- **RLS (Row Level Security, 列級安全)**：資料庫層級的權限規則，決定「哪個帳號能看到哪幾筆資料」。是本系統最核心的安全機制——例如顧問只看得到自己負責的學生。
- **migration (資料庫遷移腳本)**：修改資料庫結構或權限的程式檔，編號累加（如 0048、0049）。
- **commit / merge / tag**：git 版本控制的動作。commit=存一次檔；merge=把修改併入正式主線（main）；tag=在某個版本釘一個名牌（如 v1.0）。

## 二、四個角色與分工

這套協作的關鍵，是**四個角色各守本分、互不越界**。理解這張表，就理解了整個專案怎麼運轉。

| 角色                | 是誰                                 | 負責什麼                                                                                       | 刻意不碰什麼                                        |
|-------------------|------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------|
| 產品負責人 + 驗收者       | Marcus (放洋營運, GitHub: fangyang-op) | 提出需求、定義「要什麼」；用截圖／實際操作驗收結果； <b>唯一經手所有密碼／token／金鑰的人</b> ；做所有不可逆動作（按下 merge、刪除、在資安平台上判定）      | 不需要看懂程式碼；不需要自己寫程式                             |
| 規劃大腦 + 文件官 + 資安推理 | Claude (對話端)                       | 把口語需求整理成精確的 Markdown 施工規格 (spec) 給 CC；維護 Notion 全套文件；做資安判讀與「對抗式自我審查」（紅隊思考）；判斷每一步該不該做、做得對不對 | 不直接操作電腦／終端；不經手任何密碼或機密（不產生、不寫進文件、不寫進對話）        |
| 執行手 + 終端          | Claude Code (CC, 在 Marcus 的 Mac 上) | 依 spec 在真實終端機執行：安裝工具、跑測試、套用 migration、執行掃描器、寫／改程式、commit                                   | 不替 Marcus 決定要不要 merge；token 一律走環境變數、不印出、不寫進檔案 |
| 驗收方 + 訂標準 + 接手方   | 資訊部 (IT 工程師)                       | 接手前先發出资安檢查報告（訂出基準分數與要補的資料）；定義 11 類交付資料、P0／P1／P2 優先級、與                                      | <b>沒有參與開發</b> ——他們是定標準與驗收的一方，不是動手做的人          |

| 角色 | 是誰 | 負責什麼                 | 刻意不碰什麼 |
|----|----|----------------------|--------|
|    |    | 「接手前最低條件」；未來接手維運這個平台 |        |

## 資訊部到底扮演什麼角色？（常被誤解，特別說明）

資訊部在這個專案裡不是開發者，而是「客戶兼裁判」：

- 他們先丟出一份**初步資安檢查報告**（用 SonarQube 掃出基準：Security Review E、Coverage 0%、一堆待修項），等於畫出「要達到這裡才能交接」的紅線。
- 他們列了**11類必須補齊的資料**（系統說明、權限、資料表、個資盤點、環境變數、部署、測試帳號、掃描結果、修補紀錄.....）與**5項最低交接條件**。
- 整個專案的所有產出，都是**對著資訊部這份清單在做**——做完一項、對照一項、回報一項。
- 換句話說：資訊部定義了「成功長什麼樣」，Marcus + 兩個 AI 負責「做到那個樣子並證明給他們看」。

## 三、核心工作流程（這套怎麼運轉的）

每一個任務，都跑同一個循環：

1. **Marcus 提出需求**（口語，例如「密碼產生器好像不夠安全」「資訊部要我們補測試」）。
2. **Claude 把需求拆解、研究、整理成一份精確的 Markdown 施工規格**——寫清楚要改什麼檔、為什麼、預期結果、怎麼驗收。
3. **Marcus 把這份 spec 貼給 CC**。
4. **CC 在真實終端機執行**，把過程與結果（log、測試輸出、截圖）回報。
5. **Marcus 把 CC 的回報貼回給 Claude**。
6. **Claude 判讀結果、驗證是否真的成立，更新 Notion 文件**；若沒過，回到第 2 步修正 spec。
7. **不可逆的動作**（按 merge、輸密碼、在資安平台上判定 Safe）一律由 Marcus 親手做。

這個循環的精髓：**Claude 是大腦、CC 是手、Marcus 是握著方向盤與煞車的人**。三者之間用「貼上文字」串接，Marcus 不需要懂程式，只需要會複製貼上、會看截圖、會說「這個對／不對」。

## 一個真實例子：資安報告最痛的「Security Review E」 是怎麼變成 A 的

這個例子完整展示四個角色如何接力：

- **資訊部**的報告指出 Security Review 評級為 **E**（最差），列為頭號痛點。
- **Claude** 判讀後發現：那些被標記的「高風險點（hotspot）」其實是同一條 email 格式檢查的正規表達式，經過分析是**安全的誤報**（它結構單純、不會造成效能攻擊，輸入長度也有上限），並寫好每一條的中文說明理由。
- **CC** 在終端機跑掃描、把這些判讀寫成一份技術文件。
- **Marcus** 做了只有真人能做的那一步：到 SonarCloud 網頁，把每一個 hotspot 逐一點「標記為已複審 → 安全」，並貼上 Claude 寫好的理由。
- **結果**：複審率變 100%，Security Review **E** → **A**。

沒有任何一方能單獨完成這件事：AI 能判讀與寫理由，但「在網頁上做出安全判定」這個帶責任的動作，留給了人。

## 四、為什麼這樣分工 — 五條鐵則（本報告的教學核心）

這套協作之所以能做出「資安等級」的成果而不出包，靠的是五條反覆執行的紀律。想複製這套做法，這五條比任何工具都重要。

### 鐵則 1：真人守在所有「不可逆」的關口

AI 可以規劃、可以動手，但凡是做錯了會難以挽回的動作，一律由人親手執行：輸入密碼／token、按下 merge 到正式線、刪除資料、在資安平台上做安全判定。本專案中，密碼一律 Marcus 自己輸入、merge 一律 Marcus 在 GitHub 點、hotspot 安全與否一律 Marcus 判定。AI 提供判斷依據，人保留最後決定權。

### 鐵則 2：機密永遠不進對話、不進文件

整個專案中，沒有任何一把密碼、token、金鑰出現在 Claude 的對話或 Notion 文件裡。token 由 Marcus 在自己終端機用環境變數設定，CC 用環境變數讀取、從不印出。連截圖裡的 token 都先打碼，事後還去重新產生作廢。原則：AI 經手的是『怎麼做』，不是『鑰匙本身』。

### 鐵則 3：驗證，而不是假設

AI 工具會「以為自己成功了」。本專案養成兩個習慣：① 每次改完 Notion 文件，重新抓取頁面確認真的改到了（曾發現編輯指令在比對不符時會「靜默略過、卻回報成功」——不驗證就會以為改了其實沒改）；② 判斷程式有沒有上線時，用 git 指令確認

遠端的真實狀態，而不是相信本機可能過時的畫面。原則：**重要的事，眼見為憑、再三確認。**

## 鐵則 4：對抗式自我審查（紅隊思考）

Claude 不只「把事情做完」，還會**反過來攻擊自己的成果**：假設自己是攻擊者，這個修補有沒有漏洞？本專案的 v1.0 複審就靠這種多角度對抗審查，**額外揪出 2 個原本沒發現的權限漏洞**（顧問可以繞過介面、直接改資料庫裡的收款人＝佣金挪移等），並補上修補。原則：**AI 最有價值的不是寫得快，是願意懷疑自己。**

## 鐵則 5：每個資安主張都要有證據

文件裡不寫「我們有做 RLS」這種空話，而是寫「**這是測試結果：用顧問帳號去抓別人的學生，回傳 0 筆、被擋下**」。本專案所有安全宣稱都對應一條自動化測試或一次掃描結果（最終 91 個單元測試、跨角色權限實測、完整 git 歷史的機密掃描）。原則：**對驗收方說話，要拿得出可重複的證據，不是口頭保證。**

# 五、專案全紀錄（前因後果）

## 起點

放洋自建了一套全端 CRM（技術上是 Next.js + Supabase + Vercel，前後端都架在東京），管理學生整個生命週期。系統可運作，但屬於早期版本（接近 Beta）。公司決定由資訊部評估接手。

## 轉折：資訊部的資安報告

資訊部用 SonarQube 對系統做了初步掃描，產出一份報告：Quality Gate 雖通過，但 **Security Review E、可靠性 C、測試覆蓋率 0%、0 個測試**，並列出 42 個 Bug、1 個漏洞、361 個程式異味、8 個高風險點。同時附上**11 類要補齊的資料與接手前最低條件**。這份報告成為整個專案的施工藍圖。

## 過程：分階段強化（約兩週）

依「先止血、再補強、最後證明」的順序推進，每一步都跑第三節那個循環、由資訊部清單驗收：

1. **唯讀盤點**：先全面體檢、不動任何東西，摸清現況。
2. **止血高風險項**：密碼產生器改用密碼學等級亂數；把有 4 個高危漏洞的舊框架（Next.js 14）升級到新版（Next 15 + React 19），高危漏洞 4 → 0。
3. **收斂資料揭露**：修正「電話反查會洩漏其他顧問學生姓名」的問題（改成最小揭露）；上傳檔案加內容檢查防偽裝；CSV 匯出防公式注入；把過度抓取資料的查詢

收斂為只取必要欄位。

4. **補功能缺口**：補上忘記密碼／重設密碼的完整流程；把後台設定頁收緊為僅管理員可進。
5. **補測試**：建立自動化測試套件（單元 + 跨角色權限實測 + 端到端流程），全部進入持續整合（每次改動自動跑）。
6. **認證強化**：密碼政策拉到 12 碼 + 四類字元、改用營運信箱寄送驗證信、忘記密碼流程端到端實測通過。
7. **資料庫權限收斂**：對內部函式收回不必要的執行權限（migration 0048）。
8. **v1.0 上線前總複審 + 欄位級權限**（migration 0049）：把資料層做成「真邊界」——即使有人繞過畫面、直接對資料庫下指令改派學生／反刪除／改收款人，也會被擋下（實測回傳「權限不足」）。

## 收尾：重新檢測、證明、移交

1. 跑 **Semgrep** 深度掃描 → 0 個需處理的真實問題（唯一一個是不可利用的誤報）。
2. 建立 **SonarCloud** 專案、把測試覆蓋率接上、重新掃描。
3. Marcus 在網頁逐一把高風險點判定為安全 → Security Review **E** → **A**。
4. 把「修補前 vs 後」對照填進移交文件，所有產出對齊資訊部 11 類需求。
5. 把所有修補 merge 進正式線、部署上線，並在 git 標記 **v1.0** 版本。

## 六、最終成果（數據）

| 指標                    | 資訊部報告（修補前） | 移交時（修補後）  |
|-----------------------|------------|-----------|
| Quality Gate（總體品質閘門）  | 通過         | <b>通過</b> |
| Security（安全性）         | B          | <b>A</b>  |
| Security Review（資安複審） | <b>E</b>   | <b>A</b>  |
| Reliability（可靠性）      | C          | <b>A</b>  |
| Maintainability（可維護性） | A          | A         |
| 測試覆蓋率 Coverage        | 0.0%       | 5.6%      |
| 測試數 Tests             | 0          | <b>91</b> |
| Bug                   | 42         | <b>0</b>  |
| 漏洞 Vulnerability      | 1          | <b>0</b>  |



| 指標                    | 資訊部報告（修補前） | 移交時（修補後）    |
|-----------------------|------------|-------------|
| 高風險點 Security Hotspot | 8          | 0 待審（全判定安全） |

- **深度掃描（Semgrep）**：0 個需上線前處理的真實問題。
- **完整 git 歷史機密掃描**：零外洩。
- **交付文件**：Notion 8 份主文件 + 2 份附件子頁，涵蓋資訊部全部 11 類需求。
- **版本**：正式線已部署，git 標記 v1.0。
- **尚待會議決策（屬治理、非安全瑕疵）**：資料自動備份、是否導入登入雙重驗證（MFA）、系統帳號擁有權收歸公司、自訂網域。

## 七、FAQ 專區

### ▼ Q1：Marcus 不會寫程式，這樣做出來的系統能信嗎？

能，但關鍵不在「AI 寫得好」，而在「驗收夠嚴」。本專案的信任來自三件事：① 每個安全宣稱都有可重複的測試或掃描證據；② AI 會對抗式地攻擊自己的成果找漏洞；③ 所有不可逆的動作由真人把關。**不是因為 AI 不會錯，而是因為這套流程能接住錯。**

### ▼ Q2：為什麼要用兩個 AI（Claude 和 Claude Code）？一個不夠嗎？

因為「思考」和「動手」是兩種不同的安全模型。Claude（對話端）負責規劃與判讀，刻意**不碰你的電腦**，所以它不可能誤刪你的檔案。Claude Code 能操作真實終端機，威力大但風險也大，所以它**不替你決定要不要 merge、不碰密碼**。把兩者分開，等於在「規劃」與「執行」之間放了一道由真人（Marcus）守的閘門。

### ▼ Q3：密碼和金鑰會不會外洩給 AI？

不會。整個專案有一條鐵則：**機密永不進對話、不進文件**。密碼、token、金鑰一律由 Marcus 自己在終端機輸入；CC 透過環境變數讀取、從不印出或寫進檔案；Claude 全程不經手。連截圖裡的 token 都先打碼，事後還作廢重生。AI 知道「怎麼用鑰匙」，但從不持有鑰匙本身。

### ▼ Q4：資訊部有參與開發嗎？他們做了什麼？

資訊部沒有寫任何程式。他們扮演「客戶兼裁判」：先發出资安報告畫出「要達到哪裡才能交接」的紅線、列出要補的 11 類資料，最後接手維運。整個專案是**對著資訊部的標準在做、做完證明給他們看**。

### ▼ Q5：「Security Review E → A」是把漏洞修掉了嗎？

部分是、部分是「澄清誤判」。E 之所以是 E，主因是「有高風險點還沒有人複審」，不代表真的有危險。經分析，那些點其實是安全的誤報（一條結構單純、有長度上限的 email 檢查式）。真正讓評級上升的動作，是由真人在平台上逐一判定為安全並附上理由。同時，真正的弱點（如弱亂數）也確實在前面階段修掉了。

▼ Q6：為什麼測試覆蓋率只有 5.6%，不是更高？這樣夠嗎？

這個數字偏低但誠實。它只計算「純邏輯程式」這一層（這層覆蓋率約 94%）；系統的介面、伺服器動作、資料庫權限是由另一種測試（整合測試＋端到端測試＋跨角色權限實測）把關，那些不會被算進這個百分比。換句話說，安全關鍵的部分有測，只是沒反映在這個單一數字上。我們選擇照實呈現、並在文件裡說明，而不是為了好看去灌數字。

▼ Q7：這套「AI 協作 + 真人驗收」的模式，可以複製到別的專案嗎？

可以，而且這正是本報告的目的。複製的關鍵不是工具，是第四節那五條鐵則：真人守不可逆關口、機密不進對話、驗證而非假設、對抗式自我審查、每個主張都有證據。工具會換，紀律不變。下一節有上手清單。

▼ Q8：整個過程花了多久？最花時間的是什麼？

資安強化與檢測這一輪約兩週。最花時間的不是「寫程式」，而是驗證與證明——反覆確認修補真的生效、把證據整理成資訊部看得懂的文件、以及對抗式複審把關。這也呼應了核心觀念：在 AI 協作裡，產出很快，慢的是負責任地確認它是對的。

## 八、想複製這套做法？上手清單

給放洋內部或任何想用「Vibe coding × 真人驗收」做專案的人。

1. **先定義「成功長什麼樣」**：找出你的「資訊部清單」——一份明確、可驗收的需求／標準。沒有靶，AI 再強也只是亂槍。
2. **分清三個角色**：誰是產品負責人兼驗收者（人）、誰負責規劃與文件（對話 AI）、誰負責動手執行（終端 AI）。一個人可以同時是前者，但驗收權與機密一定留在人手上。
3. **用「貼上文字」串接**：需求 → 規格 → 執行 → 回報 → 驗證，全靠複製貼上流轉。你不需要懂程式，只要會描述需求、會看結果對不對。
4. **守住不可逆的關口**：密碼、上線（merge）、刪除、帶責任的判定，一律自己親手做。
5. **機密絕不貼進對話或文件。**



6. **凡事驗證**：AI 說做好了，自己再確認一次（重抓文件、查真實狀態）。
7. **要求 AI 攻擊自己的成果**：問「這有沒有漏洞？如果我是攻擊者會怎麼打？」
8. **要證據、不要口頭保證**：每個重要主張都對應一個可重複的測試或檢查結果。

**結語。** 放洋 CRM 的這次經驗證明：一個不寫程式的人，確實能主導一個達到業界資安標準的系統工程——前提是把 AI 當成「需要被驗收的高速產出者」，而不是「可以全盤信任的黑盒子」。真正的價值，來自人類保留的那三樣東西：**判斷力、責任、與懷疑的紀律。**

本報告為放洋全端 CRM 平台移交文件之延伸，記錄協作方法論；系統技術細節與資安現況見同層〈序～七〉與〈五、資訊部移交文件〉。